



Práctica 1: ampliación de la arquitectura de un modelo de lengua básico

Esta práctica tiene dos objetivos principales:

1. Familiarizarte con una implementación básica de un modelo de lengua (LLM). Esta parte es igual para todos los estudiantes.
2. Ampliar la arquitectura de este modelo con algunas de las técnicas que se utilizan en los modelos de lengua más avanzados. Esta parte se realizará en grupos de cuatro estudiantes de forma que cada grupo elegirá una o más técnicas para implementar y evaluar.

El primer objetivo no solo te permitirá entender la implementación de un LLM antes de extenderla, sino que te servirá para prepararte para el examen de prácticas, que contendrá preguntas sobre dicha implementación.

Código base

Nos centraremos en el código de los capítulos 2 a 6 del libro de Sebastian Raschka (2024), *Build a large language model (from scratch)* de la editorial Manning Publications. El libro puede consultarse en línea en [este enlace](#) del portal de libros electrónicos de O'Reilly al que tendrás que acceder con tu cuenta de la Universitat d'Alacant. Adicionalmente, hay un curso con vídeos para cada capítulo en [YouTube](#). El [código del libro](#) está disponible en Github. El repositorio contiene más código que el que aparece en el libro y es suficiente que te centres en los siguientes cuadernos:

- código del [capítulo 2](#),
- código del [capítulo 3](#),
- código del [capítulo 4](#),
- código del [capítulo 5](#),
- código del [capítulo 6](#).

Catálogo de ampliaciones

Como ampliaciones, el equipo ha de elegir hasta 3 elementos (dependiendo de la complejidad; véase más adelante) de los que se implementan en modelos de LLMs más recientes y que no se incluyen en el código base. Las ampliaciones se escogerán de entre las que se muestran en la [galería de Sebastian Raschka](#). Como alternativa a explorar las tarjetas de todos los modelos, podéis consultar los artículos de resumen que aparecen en la sección “Meta analysis” de la galería.

Validación de la ampliación

Es obligatorio aportar evidencias experimentales lo más rigurosas posibles que demuestren que la modificación está bien implementada y produce los efectos esperados. Dado que los estudiantes no disponen de recursos de cómputo masivos para entrenar modelos a gran escala, la validación debe abordarse mediante una o varias de las siguientes vías, aunque queda abierta la puerta a otras propuestas metodológicamente sólidas:

- Si la técnica ya existe en PyTorch o en librerías estándar consolidadas, se puede comparar numéricamente las salidas tensoriales directas ante una misma entrada arbitraria (con cierta tolerancia en los valores de coma flotante).
- Si la técnica propone una optimización estructural, se puede presentar una comparativa reproducible que demuestre la reducción efectiva en consumo de memoria o el incremento en la tasa de generación de tokens, por ejemplo.
- Si se realiza un entrenamiento desde cero, se considerarán modelos muy pequeños (pocas capas y dimensiones reducidas) para demostrar aprendizaje en tareas que converjan en minutos.
- Se se parte de un modelo más grande como GPT-2, se pueden realizar pruebas de fine-tuning en tareas muy acotadas y de convergencia rápida usando LoRA. También se puede evitar el entrenamiento mediante técnicas de *few-shot in-context learning*.

El posible entrenamiento de modelos usará conjuntos de datos reducidos, como el corpus Tiny Shakespeare o problemas de clasificación sencillos sobre LMMs como, por ejemplo, clasificación temática o de sentimiento. También puede ser interesante explorar tareas sintéticas de memoria asociativa (*associative recall*) como, por ejemplo, A: 3, B: 7, C: 1, ... QUERY B ->, donde el modelo debe dar muy alta probabilidad a 7.

Complejidad técnica y ponderación de la nota

La calificación final estará estrechamente ligada a la ambición y complejidad técnica de la modificación elegida. Existe un gradiente claro de dificultad: realizar un cambio mínimo o casi trivial (como limitarse a cambiar una función de activación aislada) difícilmente permitirá alcanzar el aprobado por sí solo; en el extremo opuesto, modificaciones de calado que obliguen a reformular la interacción entre múltiples módulos del transformer, gestionar transformaciones tensoriales complejas y aportar una batería empírica exhaustiva optarán a la máxima calificación.

Planificación

Dedica las dos primeras semanas a familiarizarte con el código base y a decidir qué ampliación implementar. La tercera y cuarta semanas se dedicarán a la implementación y la quinta a la evaluación. La sexta semana se dedicará a la escritura de la memoria.

Escritura de la memoria

Tenéis que entregar una memoria de la práctica en la que se describa la ampliación implementada, la forma de evaluarla y los resultados obtenidos. La memoria ha de seguir el [formato](#) de los artículos de la ACL (Association for Computational Linguistics). Aunque hay una versión en formato [DOCX](#), es recomendable que uses la plantilla para [LaTeX](#).

La memoria ha de tener una longitud máxima de 4 páginas, excluidas las referencias.

Entrega

Tenéis que entregar únicamente el PDF a través del [Moodle](#) de la asignatura. La fecha límite de entrega es el **22 de octubre de 2026**.